

Design Principles

Contents

What principles are for

1. [What a design principle is](#)

The everyday four

2. [KISS, Keep It Simple, Stupid](#)
3. [YAGNI, You Aren't Gonna Need It](#)
4. [DRY, Don't Repeat Yourself](#)
5. [SoC, Separation of Concerns](#)

SOLID

6. [SRP, Single Responsibility Principle](#)
7. [OCP, Open Closed Principle](#)
8. [LSP, Liskov Substitution Principle](#)
9. [ISP, Interface Segregation Principle](#)
10. [DIP, Dependency Inversion Principle](#)

The measurements underneath

11. [Coupling and cohesion](#)
12. [Composition over inheritance](#)

Encapsulation and boundaries

13. [LoD, Law of Demeter](#)
14. [Tell, Don't Ask](#)
15. [Information hiding](#)

Failure and surprise

16. [Fail Fast](#)
17. [Defensive programming](#)
18. [POLA, Principle of Least Astonishment](#)

Worked example

19. [One controller, refactored](#)

Reference

- 20. [Interview questions](#)
- 21. [Common mistakes](#)
- 22. [Quick reference](#)

1. What a design principle is

A design principle is a guideline for arranging code so it stays cheap to change. Each one names a cost that appears later, and following it spends a little effort now to avoid more effort afterwards.

They are guidelines, not laws. Two things follow from that, and both appear in interviews.

Principles conflict with each other. DRY pushes us to pull shared code into one place. YAGNI pushes us not to build anything we do not need yet. Both are correct, and in any given situation only one of them applies. Naming which cost is real in this case is the actual skill.

A principle applied where its cost never appears makes the code worse. Splitting a forty-line method that two people have read once each, into six classes with three interfaces, satisfies several principles and helps nobody. The question is never "does this follow the principle". It is "which change is this code likely to face, and does this shape make that change easier".

The one-line version

Every principle here is a different answer to the same question: what do we do so that the next change touches as little code as possible, and so that the code it touches is easy to understand on its own?

2. KISS, Keep It Simple, Stupid

Given two designs that both meet the requirement, take the one with fewer moving parts.

Simple is not the same as short. A dense one-line stream chain that takes five minutes to read is not simple. Simple means a reader can hold the whole thing in their head at once, and can predict what it does without running it.

A test that works in review: could someone who joined last week read this and explain it back, without asking us what it does? If not, the complexity needs either removing or a comment explaining why it has to be there.

Complexity that is not required is a permanent cost. Every extra layer is another thing to read, test, debug at midnight, and keep working when the language or framework moves.

3. YAGNI, You Aren't Gonna Need It

Build what the current requirement asks for. Do not build for the requirement we imagine arriving later.

```
// Speculative: one caller, one format, and an options object nobody populates.
public byte[] export(List<Row> data, String format, ExportOptions opts) {
    return switch (format) {
        case "csv" -> exportCsv(data, opts);
        case "json" -> exportJson(data, opts);
        case "xml" -> exportXml(data, opts);
        default -> throw new IllegalArgumentException("unknown format: " +
format);
    };
}

// What the requirement actually said.
public byte[] exportCsv(List<Row> data) { ... }
```

Two reasons this is worth defending in review:

- Code we do not write needs no tests, no debugging, and no maintenance.
- A guess about the future is usually wrong in detail. When the real requirement arrives it rarely matches the shape we prepared, so we pay twice: once to build the flexible version, once to reshape it.

YAGNI is not an argument against thinking ahead. It is an argument against *building* ahead. Knowing where a seam will probably be needed is useful. Adding the seam before anything needs it is the part to resist.

4. DRY, Don't Repeat Yourself

The target is duplicated **knowledge**, not duplicated text.

One rule living in two places is a real problem. When the rule changes, someone updates one copy and misses the other:

```
// In the REST controller
if (order.total().compareTo(new BigDecimal("10000")) > 0) {
    approvals.require(order);
}

// In the batch importer, three months later, now out of step
if (order.total().compareTo(new BigDecimal("5000")) > 0) {
    approvals.require(order);
}
```

That belongs in one method, because there is one business rule.

Two pieces of code that happen to look alike are a different matter. If they change for different reasons, they are not duplication, and merging them creates a shared thing that two callers now have to agree on. The next change makes one caller need slightly different behavior, so a boolean parameter appears. Then another. The abstraction ends up more expensive than the copies were.

The counterweight worth quoting

Sandi Metz: duplication is cheaper than the wrong abstraction. Removing a duplicate is easy at any time. Removing a bad abstraction means untangling every caller that now depends on it, which is why the wrong one survives for years.

A practical version, sometimes called the rule of three: wait for three occurrences before extracting. Two similar pieces of code do not yet show which parts vary and which stay fixed. The third one does.

5. SoC, Separation of Concerns

Each part of the system handles one kind of work. Reading an HTTP request is one concern. Deciding whether an order is valid is another. Writing rows to a database is a third.

The test is what a change forces. If adding a database column means editing the controller, those two concerns are mixed. If a business rule changes and only one class moves, they are separated.

This is the principle behind the layered shape most Spring services use: controller, service, repository. Each layer talks only to the next one, and the business rules in the middle know nothing about HTTP or SQL. Section 19 shows the refactor.

6. SRP, Single Responsibility Principle

The first of the five that make up **SOLID**, whose letters are the initials of sections 6 to 10.

SRP is also the most misquoted principle in the set. It is usually stated as "a class should do one thing", which is vague enough to justify almost any split.

The useful form: **a class should have one reason to change**. Robert Martin, who named it, later sharpened it to one source of change, meaning one group of people who ask for changes.

An `Invoice` class that calculates tax, renders a PDF, and emails the customer has three reasons to change: tax law, document design, and delivery. Three different groups ask for those changes, they arrive on different schedules, and each edit risks breaking the other two.

```
// Three reasons to change, in one class.
public class Invoice {
    public Money calculateTax() { ... }
    public byte[] renderPdf() { ... }
    public void email(String to) { ... }
}

// One reason each.
public class TaxCalculator { public Money taxFor(Invoice i) { ... } }
public class InvoiceRenderer { public byte[] toPdf(Invoice i) { ... } }
public class InvoiceMailer { public void send(Invoice i, String to) { ... } }
```

The counter-pressure is real: split too far and behavior that belongs together ends up scattered across classes, which is its own cost. Section 11 gives the measurement that decides it.

7. OCP, Open Closed Principle

Open for extension, closed for modification. Adding behavior should mean adding code, not editing code that already works and is already tested.

The usual symptom of a violation is a switch that grows with every new case, in more than one place:

```
// Every new payment method edits this method, and the three others like it.
public Money fee(String method, Money amount) {
    return switch (method) {
        case "card"    -> amount.percent(2.9);
        case "bank"    -> Money.ofCents(25);
        case "wallet"  -> amount.percent(1.5);
        default        -> Money.ZERO;
    };
}
```

The version that is closed to modification puts each method behind an interface, so adding one means adding a class:

```
public interface PaymentMethod {
    String name();
    Money fee(Money amount);
}

public class Card implements PaymentMethod {
    public String name() { return "card"; }
    public Money fee(Money a) { return a.percent(2.9); }
}
```

In Spring, every implementation can then be collected automatically, and a new payment method needs no edit here at all:

```
public class FeeCalculator {
    private final Map<String, PaymentMethod> methods;

    public FeeCalculator(List<PaymentMethod> all) {
        this.methods = all.stream()
            .collect(toMap(PaymentMethod::name, m -> m));
    }
}
```

Worth saying plainly in an interview: this principle is easy to over-apply. A switch with three cases that has not changed in two years is fine. The interface earns its place once the cases keep arriving, or once the same switch is repeated in several methods.

8. LSP, Liskov Substitution Principle

Named after Barbara Liskov. Anywhere the code expects a type, any subtype must work without the caller noticing. A subtype may do more, but it must not promise less.

The classic example:

```
public class Rectangle {
    protected int width, height;
    public void setWidth(int w) { this.width = w; }
    public void setHeight(int h) { this.height = h; }
    public int area() { return width * height; }
}

public class Square extends Rectangle {
    @Override public void setWidth(int w) { this.width = w; this.height = w; }
    @Override public void setHeight(int h) { this.width = h; this.height = h; }
}

// Correct for every Rectangle, and wrong for a Square.
int resize(Rectangle r) {
    r.setWidth(5);
    r.setHeight(4);
    return r.area(); // expects 20, a Square returns 16
}
```

A square is a rectangle in geometry. It is not a subtype of this `Rectangle`, because the class promises that width and height move independently, and `Square` breaks that promise.

The JDK contains a well-known violation of its own, which is a strong thing to name in an interview:

```
List<String> fixed = Arrays.asList("a", "b");
fixed.add("c");           // compiles, then throws UnsupportedOperationException
```

`Arrays.asList` and `List.of` both return a `List` that cannot accept `add`, so code written against the `List` interface fails at run time depending on which implementation it received.

Three violations to recognize by name:

- Overriding a method so it throws instead of doing the work, as above.
- Tightening what the caller must provide. If the parent accepts any string and the child rejects empty ones, code written against the parent starts failing.
- Weakening what the caller gets back. Returning `null` where the parent always returned a value moves the failure to the caller.

The practical version: the promise lives in the interface, not in the implementation. If a caller has to check which implementation it received, something has already broken.

9. ISP, Interface Segregation Principle

No caller should have to depend on methods it does not use. Many small interfaces beat one large one.

A large interface couples every caller to every method. Adding a method to it forces every implementation to change, including test doubles that only ever needed one method.

```
// Every caller depends on all eight, and every test double implements all eight.
public interface UserRepository {
    User findById(long id);
    List<User> findAll();
    void save(User u);
    void delete(long id);
    void deactivate(long id);
    List<User> search(String query);
    long count();
    void bulkImport(List<User> users);
}

// Segregated: a caller that only reads depends only on this.
public interface UserFinder { User findById(long id); }
public interface UserWriter { void save(User u); }
```

The JDK makes the argument better than any rule does. `Runnable`, `Comparable`, `Callable` and everything in `java.util.function` declare a single method each, which is exactly why they work as lambdas and compose freely.

The related convention worth stating in an interview: **the interface belongs to the layer that uses it, not the layer that implements it**. The service knows which two methods it needs, so it declares `OrderStore`, and the persistence class implements it. That is the ports and adapters arrangement, and it is what makes the next principle work.

10. DIP, Dependency Inversion Principle

High-level code should not depend on low-level detail. Both should depend on an abstraction.

Business rules are the high-level part. JPA, an SMTP server and a payment API are details. When the service imports the repository implementation directly, the rules cannot be tested or reasoned about without a database.

Inverting it means the service declares the interface it needs, and the concrete class is passed in:

```
// Declared in the service layer, named for what the service needs.
public interface OrderStore {
    long create(Order order);
}

@Service
public class OrderService {
    private final OrderStore store;

    // Constructor injection. Spring supplies the real one, a test supplies a fake.
    public OrderService(OrderStore store) {
        this.store = store;
    }
}
```

```
// The detail depends on the abstraction, not the other way round.
@Repository
public class JpaOrderStore implements OrderStore { ... }
```

The arrow now points from the persistence class toward the service, rather than the other way round. That is the inversion in the name.

Dependency injection is the mechanism, not the principle. Passing a dependency into a constructor is injection, and Spring automates it. Injection only becomes inversion when the parameter is an interface the high-level code defined for itself. A service whose constructor takes `JpaOrderStore` has injection without inversion, and still depends on the database.

Prefer constructor injection over field injection with `@Autowired`. A constructor makes the dependencies visible, allows the fields to be `final`, and lets the class be built in a test without a Spring context.

11. Coupling and cohesion

These two are the measurements that most of the other principles are trying to improve, which makes them a strong answer when an interviewer asks why a design is better.

Coupling is how much one class needs to know about another. Low coupling means we can change the inside of a class without anything else noticing.

Cohesion is how closely the parts inside one class belong together. High cohesion means everything in the file is working toward the same job.

The target is high cohesion inside, low coupling between. A `Utils` class is the standard example of failing both: nothing in it belongs together, and half the codebase imports it.

Signs of coupling that is too high:

- Changing one class reliably breaks the tests of another.
- A caller reaches through one object to get at a third (section 13).
- Two services share a database table, so neither can change the schema alone.

12. Composition over inheritance

Prefer building behavior by holding other objects over inheriting from a base class.

Inheritance couples a subclass to the internals of its parent. Change a protected field or the order in which two methods call each other, and subclasses break in ways the parent's tests never show. This is the fragile base class problem. Inheritance also fixes the relationship when the class is written, and Java allows only one superclass, so the decision cannot be revisited cheaply.

The JDK has a famous example of getting this wrong:

```
Stack<String> s = new Stack<>();    // Stack extends Vector
s.push("a");
s.push("b");
s.add(0, "c");    // legal, and it inserts underneath the stack
s.remove(1);      // legal, and it removes from the middle
```

Because `Stack` extends `Vector`, it exposes every `Vector` method, and a stack that can be modified in the middle is no longer a stack. `Properties` extends `Hashtable` has the same problem. Composition would have prevented both:

```
public class Stack<T> {
    private final List<T> items = new ArrayList<>(); // held, not inherited

    public void push(T item) { items.add(item); }
    public T pop()           { return items.remove(items.size() - 1); }
    public boolean isEmpty() { return items.isEmpty(); }
}
```

Only the three methods we chose are public. The list is an implementation detail, and it can be swapped without any caller noticing.

The guidance for Java, where both are available: use inheritance for a genuine "is a" relationship with a base that is not going to move, and composition for everything else. Joshua Bloch's rule in *Effective Java* is stronger, and worth quoting: design and document for inheritance, or else prohibit it by making the class `final`.

13. LoD, Law of Demeter

Also called the principle of least knowledge, which describes it better. Talk only to immediate collaborators. A method should call methods on its own fields, its parameters, and objects it created, and not go further.

```
// Reaches through three types. Now coupled to all of them.
String city = order.getCustomer().getAddress().getCity();

// Asks the object it already has.
String city = order.shippingCity();
```

The first version breaks when `Address` changes shape, even though the calling code cares about nothing except the city. It also makes the call impossible to test without building a customer and an address.

It is a guideline rather than a rule. Chained calls on a builder or a stream are fine, because those are one thing presenting a fluent surface, not three separate types:

```
// Not a violation. Every call returns the same builder.
Order order = Order.builder().userId(42).total(TEN).build();
```

14. Tell, Don't Ask

Tell an object what to do. Do not ask it for its data and then decide on its behalf. Behavior belongs with the data it works on.

```
// Ask: the caller pulls the state out and applies the rule elsewhere.
if (account.getBalance().compareTo(amount) >= 0) {
    account.setBalance(account.getBalance().subtract(amount));
} else {
    throw new InsufficientFundsException(account.getId());
}

// Tell: the rule lives with the data it protects.
account.withdraw(amount);
```

The first version has a problem that grows quietly. That rule is now in the caller, so the next caller writes it again, slightly differently, and section 4 explains what happens next. It also means `Account` cannot defend its own balance, because any caller can set it to anything.

The anti-pattern has a name worth knowing: Martin Fowler calls a class with only getters and setters, and all the rules living in a service, an **anemic domain model**. It is common in Java because frameworks encourage the shape, and an interviewer who asks about it wants to hear that data and the rules over that data belong together.

This is close to the Law of Demeter but not the same. LoD is about how far a caller reaches. Tell, Don't Ask is about whether the caller should be making the decision at all. Both push behavior toward the data.

It is a default rather than an absolute. Query methods that return data are fine, and objects crossing a boundary as DTOs or being serialized to JSON are meant to be plain. The target is the rule, not every getter.

15. Information hiding

David Parnas set this out in 1972: a module should hide the design decisions most likely to change, so that when they change, only that module moves.

Encapsulation and information hiding are not the same thing, and the difference is a good interview answer. Encapsulation is the mechanism, `private` fields and public methods. Information hiding is the principle that decides what to hide and why. A class with a private field and a matching getter and setter for each one has used encapsulation and hidden nothing.

```

// Hides nothing. The list is public through a pair of methods, and any
// caller can mutate it or replace it.
public class Order {
    private List<Item> items;
    public List<Item> getItems()          { return items; }
    public void setItems(List<Item> items) { this.items = items; }
}

// Hides the decisions: which collection type, and how a total is computed.
public class Order {
    private final List<Item> items = new ArrayList<>();

    public void add(Item item) { items.add(item); }
    public List<Item> items()  { return List.copyOf(items); }
    public Money total()      { return items.stream()
                                .map(Item::price)
                                .reduce(Money.ZERO, Money::plus); }
}

```

The second version can swap `ArrayList` for something else, add a discount rule inside `total`, or start rejecting duplicate items, and no caller changes. `List.copyOf` returns an unmodifiable list, so a caller cannot reach in and add an item behind the object's back.

The question to ask about any class: which decision inside it is most likely to change, and can that change happen without editing anything else? What is likely to change is what needs hiding.

16. Fail Fast

Detect a problem at the earliest point and stop there, rather than carrying on with bad state. A failure at the source is cheap to diagnose. The same failure discovered three days later, as wrong numbers in a report, is not.

```

public OrderService(OrderStore store, Notifier notifier) {
    this.store = Objects.requireNonNull(store, "store");
    this.notifier = Objects.requireNonNull(notifier, "notifier");
}

public record Order(long userId, BigDecimal total, String email) {
    public Order {
        if (total == null || total.signum() <= 0) {
            throw new IllegalArgumentException("total must be positive");
        }
        if (email == null || email.isBlank()) {
            throw new IllegalArgumentException("email is required");
        }
    }
}

```

Validating in the record's compact constructor means an invalid `Order` cannot exist at all. Every method that receives one can then trust it, which removes checks from everywhere else.

The same argument applies to startup. Missing configuration should stop the application from starting, not surface as a null at three in the morning on the first request that needs it.

What fail fast rules out: returning `null` on bad input, quietly substituting a default, and catching an exception only to log it and carry on. Each of those moves the failure away from its cause, and the stack trace then points at the wrong place.

17. Defensive programming

Write code that survives inputs it did not expect. Validate parameters, guard against nulls, handle states that should not happen.

Taken too far, this is the principle that fights the previous one, and the interview question is how to hold both:

```
// Defensive to the point of hiding bugs.
public Money total(Order order) {
    if (order == null) return Money.ZERO;
    if (order.items() == null) return Money.ZERO;
    ...
}
```

A null order is somebody's bug. Returning zero does not fix it, it converts a loud exception into a quietly wrong number in a financial report, which is a much worse outcome.

The way to settle it is by **trust boundary**:

- **At the boundary**, be defensive. HTTP requests, message payloads, file contents and calls from other services are outside our control, so validate all of it and reject what fails. In Spring that is `@Valid` with bean validation on the request object.
- **Inside the boundary**, fail fast. If an invariant is broken there, it is our own bug, and `Objects.requireNonNull` at the top of the method makes it visible immediately.

One narrower use of the term is always correct, and Joshua Bloch makes the case in *Effective Java*: **defensive copies**. Copy a mutable parameter on the way in and a mutable field on the way out, so a caller cannot change our state after the fact.

```
public final class Period {
    private final Date start;

    public Period(Date start) {
        this.start = new Date(start.getTime()); // copy in
    }

    public Date start() {
        return new Date(start.getTime()); // copy out
    }
}
```

18. POLA, Principle of Least Astonishment

Also called the principle of least surprise. A component should behave the way a reasonable reader expects from its name and signature. When behavior surprises, either the name is wrong or the behavior is.

Violations worth recognizing:

```
// A getter that mutates and writes to the database.
public User getUser(long id) {
    User u = repo.load(id);
    u.setLastAccessed(Instant.now());
    repo.save(u);
    return u;
}

// A boolean parameter the caller cannot read.
orders.process(order, true); // true meaning what?

// Returning null where a caller reasonably expects an empty list.
public List<Item> items() { return items.isEmpty() ? null : items; }
```

Each one costs a reader a trip into the body of the method to find out what it does, which is the tax the principle is trying to avoid.

Other common ones: `equals` without `hashCode`, so the object misbehaves in a `HashMap`. A method named `validate` that also saves. A method that sometimes throws and sometimes returns an error code.

The JDK has its own examples, which are useful to name in an interview because they are familiar.

`Arrays.asList` returns a fixed-size list, so `add` throws even though the type says `List`.

`String.replace` takes plain text while `String.replaceAll` takes a regular expression, and nothing in either name says so.

The test: can a reader predict what a method does from its signature alone, without opening it? If not, rename it, or change it to do the obvious thing.

19. One controller, refactored

The starting point, with three concerns in one method:

```
@RestController
public class OrderController {

    private final JdbcTemplate jdbc;
    private final JavaMailSender mail;

    @PostMapping("/orders")
    public ResponseEntity<CreateOrderResponse> create(@RequestBody CreateOrderRequest
req) {
        if (req.total().signum() <= 0) {
            return ResponseEntity.badRequest().build();
        }

        Long id = jdbc.queryForObject(
            "INSERT INTO orders (user_id, total) VALUES (?, ?) RETURNING id",
            Long.class, req.userId(), req.total());

        SimpleMailMessage msg = new SimpleMailMessage();
        msg.setTo(req.email());
        msg.setText("Order " + id + " confirmed");
        mail.send(msg);

        return ResponseEntity.ok(new CreateOrderResponse(id));
    }
}
```

What is wrong with it, in the language an interviewer wants:

- Three reasons to change: the HTTP contract, the business rule, the storage schema. That is SRP.
- It depends on `JdbcTemplate` and `JavaMailSender` directly, so the business rule cannot run without a database and a mail server. That is DIP.
- The rule `total > 0` is buried between transport code and SQL, where nobody looks for it.

The service, which knows nothing about HTTP or SQL:

```

public interface OrderStore {
    long create(Order order);
}

public interface Notifier {
    void orderConfirmed(String email, long orderId);
}

@Service
public class OrderService {

    private static final Logger log = LoggerFactory.getLogger(OrderService.class);

    private final OrderStore store;
    private final Notifier notifier;

    public OrderService(OrderStore store, Notifier notifier) {
        this.store = store;
        this.notifier = notifier;
    }

    public long create(Order order) {
        if (order.total().signum() <= 0) {
            throw new InvalidTotalException("total must be positive");
        }

        long id = store.create(order);

        try {
            notifier.orderConfirmed(order.email(), id);
        } catch (NotificationException e) {
            // A failed notification does not undo a placed order.
            log.warn("notify order {}: {}", id, e.getMessage());
        }
        return id;
    }
}

```

The controller now does only transport work:


```

@RestController
public class OrderController {

    private final OrderService orders;

    public OrderController(OrderService orders) {
        this.orders = orders;
    }

    @PostMapping("/orders")
    public CreateOrderResponse create(@RequestBody @Valid CreateOrderRequest req) {
        return new CreateOrderResponse(orders.create(req.toOrder()));
    }

    @ExceptionHandler(InvalidTotalException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ErrorResponse handleInvalidTotal(InvalidTotalException e) {
        return new ErrorResponse(e.getMessage());
    }
}

```

Six principles moved at once: SoC, SRP, DIP, ISP (each interface names one method the service needs), OCP (a second notifier is a new class, not an edit here), and fail fast (the total is rejected before anything is written, and the exception is unchecked so Spring rolls the transaction back).

The gain that matters most is testability. `OrderService.create` can now be tested with two fakes, in milliseconds, with no database and no Spring context:

```

@Test
void rejectsNonPositiveTotal() {
    var service = new OrderService(order -> 1L, (email, id) -> {});
    assertThrows(InvalidTotalException.class,
        () -> service.create(new Order(42, BigDecimal.ZERO, "a@b.com")));
}

```

Both interfaces have one method, so both fakes are lambdas. That is ISP paying for itself. If a refactor does not make something easier to test, it is worth asking what it did make easier.

20. Interview questions

Each answer below is written to be said out loud as it stands. The last sentence always leads toward the next question rather than closing the topic.

"What does SOLID stand for?"

Five principles for arranging code so it stays cheap to change, and the letters are the initials of the five.

- **S, Single Responsibility Principle.** A class should have one reason to change, meaning one group of people who ask for changes to it.
- **O, Open Closed Principle.** Open for extension, closed for modification. Adding behavior should mean adding a class rather than editing code that already works.
- **L, Liskov Substitution Principle.** Any subtype has to work anywhere the base type is expected, without the caller knowing which one it received.
- **I, Interface Segregation Principle.** No caller should depend on methods it does not use, so many small interfaces beat one large one.
- **D, Dependency Inversion Principle.** High-level code depends on an abstraction it defines, and the low-level detail implements it.

The two that matter most in practice are single responsibility and dependency inversion, because those are the ones that decide whether the code can be tested without a database.

"What is the Single Responsibility Principle, really?"

The common version, "a class should do one thing", is too vague to be useful, because "one thing" can be argued at any size. The version worth using is that a class should have one reason to change. An invoice class that calculates tax, renders a PDF and emails the customer has three reasons: tax law, document design, and delivery. Those come from three different groups of people, on three different schedules, and each edit risks breaking the other two. So the question to ask about any class is who asks for changes to it, and whether that is one group or several.

"How do you decide when to add an abstraction?"

The pressure runs both ways, so it is a judgment rather than a rule. DRY, don't repeat yourself, says pull the shared thing into one place. YAGNI, you aren't gonna need it, says do not build what is not needed yet. What settles it is whether the repeated code is duplicated knowledge or duplicated text. If one business rule lives in two places, that is a real problem, because the next change updates one and misses the other. If two pieces of code just look alike but change for different reasons, merging them creates a shared thing that both callers now have to agree on, and the boolean parameters start arriving. My default is the rule of three, waiting for the third occurrence, because two examples do not yet show which parts vary. Sandi Metz put it well: duplication is cheaper than the wrong abstraction, since removing a duplicate is easy and removing a bad abstraction means untangling every caller.

"Give an example of a Liskov violation."

The classic one is a square inheriting from a rectangle. The rectangle promises that width and height move independently, so a method that sets width to five, sets height to four, and expects an area of twenty is correct for every rectangle. A square keeps its sides equal, so it returns sixteen and the caller is wrong through no fault of its own. There is a real one in the JDK too, which is worth naming:

`Arrays.asList` and `List.of` both return a `List` whose `add` method throws `UnsupportedOperationException`, so code written against the `List` interface fails at run time depending on which implementation it was handed. The test is whether a caller ever has to check which implementation it received.

"What is the difference between dependency injection and dependency inversion?"

Injection is the mechanism and inversion is the principle. Passing a dependency into a constructor instead of building it inside is injection, and Spring automates that part. It only becomes inversion when the thing being passed is an interface that the high-level code defined for its own needs, rather than a concrete class from a lower layer. So if the service constructor takes a `JpaOrderStore`, that is injection without inversion, and the service still depends on the database. If the service declares a small `OrderStore` interface with the two methods it needs, and the JPA class implements it, the dependency arrow now points from the persistence layer toward the business rules. I would also use constructor injection rather than field injection with `@Autowired`, because it makes the dependencies visible, lets the fields be final, and allows the class to be built in a test with no Spring context.

"Composition or inheritance?"

Composition by default, inheritance for a genuine "is a" relationship with a base that is not going to move. The problem with inheritance is that a subclass depends on decisions the parent never promised to keep, like the order in which two methods call each other, so a change inside the parent breaks subclasses in ways the parent's own tests do not catch. That is the fragile base class problem, and Java makes it worse by allowing only one superclass. The JDK has a well-known example: `Stack` extends `Vector`, so a stack exposes `add` at an index and `remove` from the middle, which means it is no longer a stack. Composition would have exposed only push and pop. Joshua Bloch's rule in *Effective Java* is the one I follow: design and document for inheritance, or else make the class final.

"What is Tell, Don't Ask?"

It means telling an object what to do instead of asking it for its data and applying the rule ourselves. So rather than reading the balance off an account, comparing it and setting a new one, the caller says `account.withdraw(amount)` and the account enforces its own rule. The reason it matters is that a rule applied outside the object gets rewritten by the next caller, slightly differently, and the object can no longer defend its own state because any caller can set it to anything. The anti-pattern name is worth using here: Martin Fowler calls a class with only getters and setters, and all the logic in a service, an anemic domain model. It is common in Java because the frameworks encourage that shape. I treat it as a default rather than an absolute, since a DTO crossing a boundary is meant to be plain data.

"How do you decide between failing fast and defensive programming?"

By trust boundary, because the two pull in opposite directions and both are right in their own place. Outside the boundary, anything from an HTTP request, a message payload or another service is not under our control, so I validate all of it and reject what fails, which in Spring is bean validation on the request object. Inside the boundary I fail fast instead, because a broken invariant there is our own bug, and `Objects.requireNonNull` at the top of a method makes it visible at the point it happened. What I try to avoid is defensive code that returns a default for bad input, since a method that returns zero for a null order turns a loud exception into a quietly wrong number in a report, which is far harder to find. The one form of defensiveness that is always right is defensive copies of mutable parameters and return values, from *Effective Java*.

"Which principle do you break most often, and why?"

DRY, deliberately, and usually early in a feature. When something appears twice I copy it rather than extract it, because two examples do not show me which parts are the real shared rule and which are coincidence. Extracting at that point tends to produce an abstraction that grows a boolean parameter as soon as the third caller arrives. So I let it sit until the third use, and then the shape is obvious. The other one I relax is the open closed principle, since a switch with three cases that has not changed in two years does not need an interface. I add that seam when the cases start arriving, not before.

21. Common mistakes

- Treating the principles as a checklist to satisfy rather than costs to weigh. Code can follow all five SOLID principles and still be hard to change, usually because it was split along the wrong lines.
- Quoting DRY to justify merging code that is similar today but changes for different reasons. The boolean parameters that follow are the sign it was the wrong call.
- Splitting one clear forty-line method into six classes because of SRP. Cohesion is a cost too, and reading six files to follow one flow is worse than reading one method.
- Building the interface, the factory and the registry before there is a second implementation. That is YAGNI, and the shape rarely matches the second implementation when it finally arrives.
- Declaring an interface next to its implementation rather than in the layer that calls it. That points the dependency arrow the wrong way and usually produces one large interface instead of several small ones.
- Field injection with `@Autowired` on private fields. It hides the dependency list, prevents final fields, and forces a Spring context in tests that should not need one.
- Adding a `Utils` or `Common` class. Nothing in it belongs together, everything imports it, and it fails cohesion and coupling at the same time.
- Generating a getter and a setter for every private field and calling it encapsulation. That publishes the field through two methods and hides no decision at all.
- Catching an exception, logging it, and continuing. The failure now happens somewhere else, with a stack trace pointing at the wrong place.

- Writing a null check without deciding whose bug it is guarding against. At a trust boundary it is validation and belongs there. Inside, it is hiding a bug that should have stopped the request.
- Answering an interview question with only the definition. The follow-up is always about the trade, so name the counter-pressure before it is asked for.

22. Quick reference

Principle	Full form	One line
KISS	Keep It Simple, Stupid	Of two designs that work, take the one with fewer moving parts
YAGNI	You Aren't Gonna Need It	Build for the current requirement, not the imagined one
DRY	Don't Repeat Yourself	One piece of knowledge lives in one place
Rule of three	—	Wait for the third occurrence before extracting
SoC	Separation of Concerns	Transport, business rules and storage are different jobs
SRP	Single Responsibility Principle	One reason to change, meaning one group asking for changes
OCP	Open Closed Principle	Add behavior by adding a class, not by editing working code
LSP	Liskov Substitution Principle	A subtype works anywhere the base type is expected
ISP	Interface Segregation Principle	No caller depends on methods it does not use
DIP	Dependency Inversion Principle	High-level code defines the interface, detail implements it
LoD	Law of Demeter	Talk to immediate collaborators, not through them
POLA	Principle of Least Astonishment	Behave the way the name and signature suggest
—	Tell, Don't Ask	Tell an object what to do rather than pulling its data out
—	Information hiding	Hide the decisions most likely to change
—	Fail Fast	Stop at the first sign of bad state, not three days later
—	Defensive programming	Validate at the trust boundary, fail fast inside it
—	Coupling	How much one class must know about another. Keep it low
—	Cohesion	How well the parts inside a class belong together. Keep it high

Symptom	Principle it points at
The same switch statement in four methods	OCP
A test needs a real database or a Spring context	DIP
One business rule written in two places	DRY
A test double implements six unused methods	ISP

Symptom	Principle it points at
<code>a.getB().getC().getD()</code> in a caller	LoD
Changing one class breaks another class's tests	Coupling
A <code>Utils</code> class everything imports	Cohesion
An override that throws <code>UnsupportedOperationException</code>	LSP
A config option with one caller and one value	YAGNI
A class of only getters and setters, rules in a service	Tell, Don't Ask
A getter and setter for every private field	Information hiding
A method returning a default for invalid input	Fail Fast
A null check that hides whose bug it is	Defensive programming
<code>process(order, true)</code> at a call site	POLA
A getter that writes to the database	POLA